

BOOTCAMP ARQUITETO(A) DE SOFTWARE

Desafio Final: Partner Products API

Arquitetura MVC, API REST e persistência implementadas em JavaScript

AUTOR	Ronoel Lima
STACK	Node.js 24 Express 5 SQLite Zod
ENTREGA	19 de setembro de 2026
STATUS	Implementação funcional - 6 testes aprovados

github.com/ronoellima10-ship-it/desafio-final-arquiteto-software-api

1. Visão geral e conformidade

Rastreabilidade direta entre o enunciado, a implementação e os arquivos entregues.

Resultado

Foi construída uma API REST de Produtos seguindo MVC, com todos os endpoints pedidos, persistência SQLite, documentação OpenAPI, testes automatizados, Docker e integração contínua. Os três entregáveis obrigatórios estão consolidados neste PDF e o diagrama permanece editável no draw.io.

Requisito do enunciado	Implementação	Evidência	Status
CRUD completo	POST, GET, PUT/PATCH e DELETE	src/routes e testes E2E	Atendido
Contagem	GET /api/v1/products/count	Controller + Service + Repository	Atendido
Find All	GET /api/v1/products	Listagem ordenada	Atendido
Find By ID	GET /api/v1/products/:id	404 quando inexistente	Atendido
Find By Name	GET /api/v1/products/name/:name	Busca parcial case-insensitive	Atendido
Padrão MVC	Model, Controller e representação JSON	Diagrama e estrutura de código	Atendido
Desenho arquitetural	C4 + componentes MVC	draw.io e PDF de diagramas	Atendido
Estrutura e explicação	Árvore e matriz de responsabilidades	Seções 5 e 6	Atendido
Código funcional (opcional)	Node.js + Express	Repositório GitHub	Incluído
Persistência (opcional)	SQLite com constraints	Banco criado automaticamente	Incluído

Entregáveis finais: este relatório, o PDF dos diagramas, o arquivo arquitetura.drawio e o repositório com o código executável.

2. Contexto, escopo e requisitos

A solução publica dados de produtos de uma empresa de vendas on-line para parceiros externos.

Problema. Parceiros precisam consultar e manter dados de produtos por uma interface pública, previsível e documentada. A aplicação deve separar interface HTTP, regras de negócio e persistência para facilitar evolução e manutenção.

Escopo. O domínio escolhido é Produto. A primeira versão administra cadastro, descrição, preço, estoque e situação ativa. Autenticação, paginação e banco distribuído ficam fora do escopo acadêmico inicial.

Requisitos funcionais

ID	Requisito	Critério de aceite
RF-01	Criar produto	Retorna 201, Location e produto persistido
RF-02	Listar todos	Retorna 200, coleção e total da resposta
RF-03	Buscar por ID	Retorna 200 ou 404 padronizado
RF-04	Buscar por nome	Pesquisa parcial sem diferenciar caixa
RF-05	Atualizar	Aceita PUT/PATCH e preserva campos não enviados
RF-06	Excluir	Retorna 204 ou 404
RF-07	Contar	Retorna o número total de produtos

Requisitos não funcionais

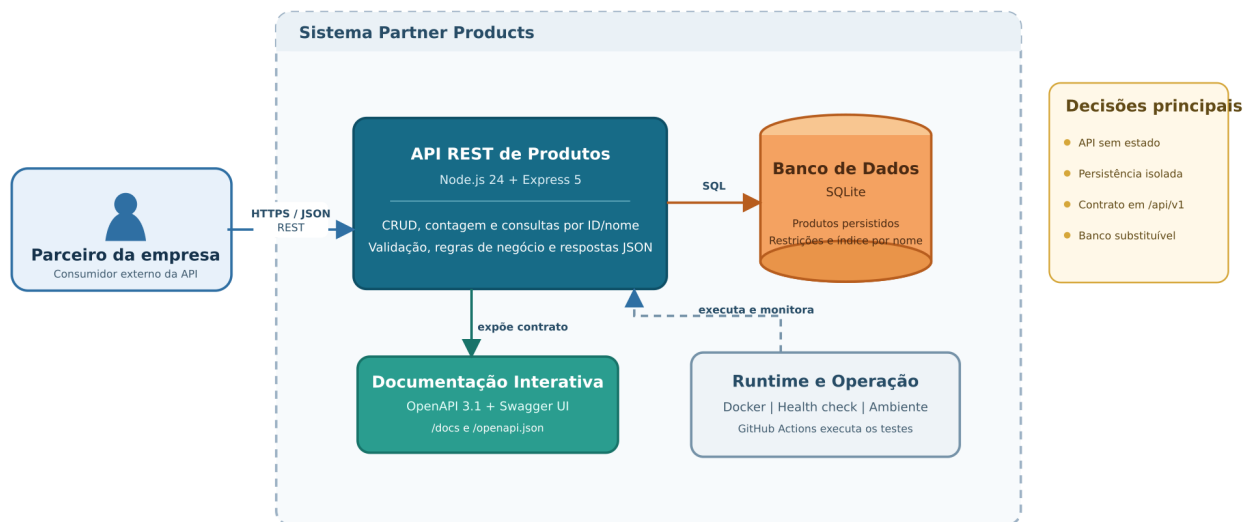
ID	Atributo	Decisão verificável
RNF-01	Manutenibilidade	Camadas coesas, dependências unidirecionais e Repository
RNF-02	Confiabilidade	Validação, constraints SQLite e encerramento controlado
RNF-03	Segurança básica	Helmet, CORS configurável, JSON limitado e sem detalhes internos em produção
RNF-04	Observabilidade	Health check e identificador por requisição
RNF-05	Portabilidade	Execução local ou Docker com variáveis de ambiente
RNF-06	Testabilidade	SQLite em memória e testes de integração automatizados

3. Arquitetura do software - C4

Visão de contêineres do sistema e suas relações em tempo de execução.

Partner Products API - C4 Nível 2: Contêineres

Visão de execução, responsabilidades e integrações



Desafio Final - Bootcamp Arquiteto(a) de Software | Arquivo-fonte editável: docs/arquitetura.drawio

Figura 1 - C4 Nível 2: parceiro, API REST, persistência, documentação e operação.

Elemento	Tecnologia	Responsabilidade
Parceiro	Cliente HTTP	Consome operações REST via HTTPS e JSON
API REST	Node.js + Express	Expõe o contrato, coordena casos de uso e devolve representações
Banco	SQLite	Persiste produtos, unicidade do nome e integridade dos valores
Documentação	OpenAPI + Swagger UI	Torna endpoints e schemas exploráveis
Operação	Docker + CI	Empacota, verifica saúde e executa testes automaticamente

Decisão de desacoplamento

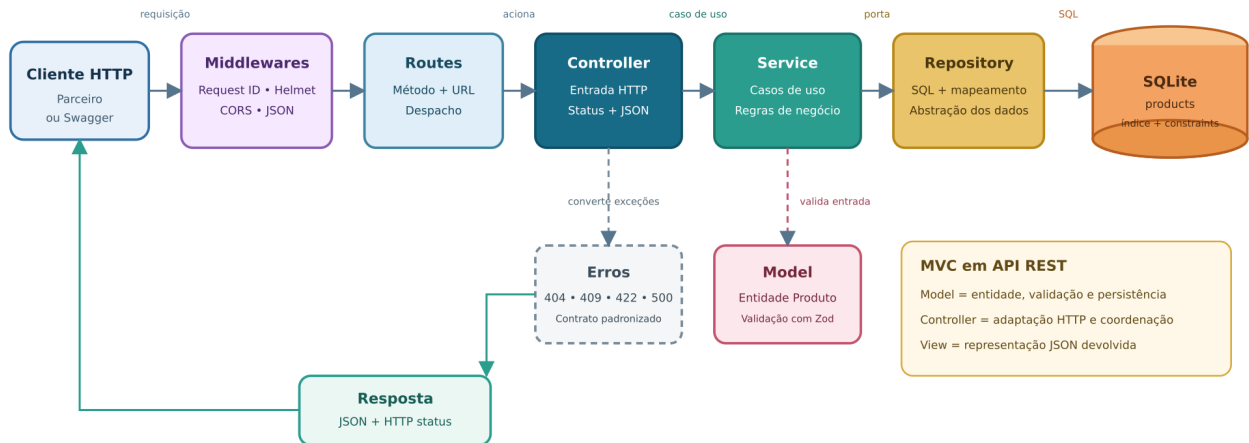
A API não acessa SQLite a partir do Controller. O Repository é a fronteira da persistência. Assim, migrar para PostgreSQL exige uma nova implementação de repositório, preservando casos de uso e contrato HTTP.

4. Arquitetura MVC e fluxo da requisição

Detalhamento dos componentes internos e dependências permitidas.

Partner Products API - Componentes MVC

Fluxo de uma requisição e separação de responsabilidades



Desafio Final - Bootcamp Arquiteto(a) de Software | Arquivo-fonte editável: docs/arquitetura.drawio

Figura 2 - Fluxo HTTP pelos componentes do padrão MVC adaptado à API REST.

Como o MVC aparece nesta API

Model: entidade, validação e persistência de Produto. Controller: traduz HTTP para chamadas de caso de uso e define a resposta. View: representação JSON entregue ao parceiro. Service e Repository refinam a separação para manter regras e detalhes de infraestrutura fora do Controller.

Direção das dependências

- Routes conhece Controller; Controller conhece Service; Service conhece Repository.
- Repository conhece a conexão SQLite, mas nenhum componente anterior executa SQL.
- Middlewares tratam preocupações transversais sem misturá-las às regras do domínio.
- Erros de aplicação são convertidos em um contrato HTTP uniforme no limite externo.

5. Estrutura de pastas MVC

Organização física do projeto e localização de cada responsabilidade.

```
desafio-final-arquiteto-software-api/
|-- .github/workflows/ci.yml
|-- data/.gitkeep
|-- docs/
|   |-- arquitetura.drawio
|   |-- diagramas/
|   |-- diagramas-arquitetura.pdf
|   |-- entregavel-final-arquitetura-software.pdf
|-- scripts/
|   |-- generate_report.py
|   |-- seed.js
|-- src/
|   |-- config/           # Banco e contrato OpenAPI
|   |-- controllers/      # Entrada e saída HTTP
|   |-- errors/           # Erros da aplicação
|   |-- middlewares/      # Segurança, contexto e erros
|   |-- models/           # Entidade e validação
|   |-- repositories/     # SQL e persistência
|   |-- routes/           # Endpoints REST
|   |-- services/         # Casos de uso e regras
|   |-- app.js            # Composição das dependências
|   |-- server.js         # Processo HTTP
|-- tests/               # Integração ponta a ponta
|-- Dockerfile
|-- compose.yaml
|-- package.json
|-- README.md
```

Princípios aplicados

Princípio	Aplicação no projeto
Responsabilidade única	Cada pasta expressa uma razão principal de mudança
Separação de interesses	HTTP, negócio, validação e dados permanecem isolados
Inversão por composição	app.js cria e injeta Repository, Service e Controller
Baixo acoplamento	O Controller não conhece SQL nem detalhes do SQLite
Alta coesão	Regras do domínio de Produto ficam centralizadas no Service e Model

6. Componentes e responsabilidades

Explicação dos elementos que compõem o código, conforme solicitado no enunciado.

Componente	Arquivo principal	Responsabilidade	Não deve fazer
Route	product.routes.js	Mapear método/URL e encaminhar	Implementar regra ou SQL
Controller	product.controller.js	Ler request, chamar Service e formar response	Conhecer banco
Service	product.service.js	Executar casos de uso, validar e tratar conflitos	Manipular HTTP
Model	product.model.js	Definir contrato e validar Produto	Responder requisições
Repository	product.repository.js	Executar SQL e mapear linhas	Definir status HTTP
Database	database.js	Conectar, criar tabela, índice e constraints	Conter regra de fluxo
Middleware	error.middleware.js	Padronizar erros e preocupações transversais	Conter caso de uso
Composition Root	app.js	Montar dependências e pipeline Express	Misturar regra de domínio
Server	server.js	Abrir porta e encerrar recursos	Construir endpoints

Exemplo de colaboração

No POST /products, a Route seleciona create; o Controller recebe o corpo; o Service valida e impede nome duplicado; o Repository executa INSERT; o Controller retorna 201 e o cabeçalho Location. Nenhuma camada assume responsabilidades das demais.

Composição das dependências

```
db -> ProductRepository -> ProductService -> ProductController
      |
      `--> ProductRouter -> Express
```

7. Modelo de domínio e persistência

Contrato de Produto, invariantes e decisões de armazenamento.

Campo	Tipo	Regra	Persistência
id	UUID	Gerado pela aplicação e imutável	TEXT PRIMARY KEY
name	String	2 a 120 caracteres; único sem diferenciar caixa	TEXT UNIQUE COLLATE NOCASE
description	String	Opcional; até 500 caracteres	TEXT NOT NULL DEFAULT ""
price	Number	Finito; entre 0 e 999.999.999,99	REAL CHECK price >= 0
stock	Integer	Entre 0 e 1.000.000	INTEGER CHECK stock >= 0
active	Boolean	Padrão true	INTEGER CHECK active IN (0,1)
createdAt	ISO 8601	Definido na criação	TEXT NOT NULL
updatedAt	ISO 8601	Atualizado a cada alteração	TEXT NOT NULL

Regras de negócio

- Um nome identifica comercialmente um produto e não pode se repetir, mesmo com variação de maiúsculas.
- Preço e estoque nunca podem ser negativos; estoque deve ser inteiro.
- Atualização parcial exige pelo menos um campo reconhecido.
- Leitura, atualização ou exclusão de ID inexistente retorna NOT_FOUND.
- Pesquisa por nome usa correspondência parcial e escapa curingas de SQL LIKE.

Persistência como diferencial

O banco é criado automaticamente na primeira execução. Em testes, a mesma implementação usa SQLite em memória. Em produção acadêmica, o arquivo fica em volume Docker. A integridade é aplicada tanto no Model quanto no banco.

8. Contrato da API REST

Endpoints versionados e semântica HTTP previsível para os parceiros.

Método	Endpoint	Sucesso	Finalidade
GET	/health	200	Verificar disponibilidade
GET	/api/v1/products	200	Listar todos os produtos
GET	/api/v1/products/count	200	Contar registros
GET	/api/v1/products/:id	200	Buscar por ID
GET	/api/v1/products/name/:name	200	Pesquisar por nome
POST	/api/v1/products	201	Criar produto
PUT	/api/v1/products/:id	200	Atualizar produto
PATCH	/api/v1/products/:id	200	Atualizar parcialmente
DELETE	/api/v1/products/:id	204	Excluir produto
GET	/docs	200	Abrir Swagger UI
GET	/openapi.json	200	Obter contrato OpenAPI

Convenções de resposta

```
{
  "data": { "id": "...", "name": "Notebook Pro 14", "price": 5499.90 }
}

{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Os dados enviados são inválidos.",
    "details": [{ "field": "price", "message": "O preço não pode ser negativo." }],
    "requestId": "..."
  }
}
```

Códigos de erro: 404 para recurso/rota ausente; 409 para conflito de nome; 422 para entrada inválida; 500 para falha inesperada. Toda resposta recebe x-request-id para correlação.

9. Fluxos principais

Sequências essenciais do comportamento da aplicação.

Criar produto

Passo	Componente	Ação
1	Cliente	Envia POST com JSON
2	Middlewares	Atribuem request ID, segurança, CORS e fazem parse
3	Route/Controller	Selecionam a operação e adaptam a entrada
4	Service/Model	Validam campos e consultam duplicidade
5	Repository/SQLite	Persistem dentro das constraints
6	Controller	Retorna 201, Location e representação JSON

Pesquisar por nome

Passo	Ação e resultado
1	GET /api/v1/products/name/teclado
2	Service valida o termo; Repository escapa %, _ e barra invertida
3	SQLite executa LIKE com COLLATE NOCASE
4	API retorna 200, lista e metadado total, inclusive quando a lista é vazia

Mapeamento de exceções

Origem	Exceção	Resposta
Model/Zod	Campo inválido	422 VALIDATION_ERROR
Service	Produto ausente	404 NOT_FOUND
Service	Nome duplicado	409 CONFLICT
Router	Endpoint ausente	404 ROUTE_NOT_FOUND
Infraestrutura	Erro inesperado	500 INTERNAL_ERROR sem vaziar detalhes

Resultado arquitetural

O caminho feliz e as falhas atravessam as mesmas fronteiras bem definidas. Isso reduz duplicação, torna o contrato previsível e permite testar o comportamento completo sem iniciar um processo HTTP externo.

10. Qualidade, segurança e operação

Mecanismos implementados para tornar a entrega reproduzível e verificável.

Área	Mecanismo	Benefício
Validação	Zod + strict schemas	Rejeita campos desconhecidos e valores fora do domínio
Persistência	Constraints + índice	Protege integridade mesmo fora do fluxo HTTP
Segurança	Helmet, CORS, limite 32 KB	Reduz superfície de configuração insegura
Erros	Envelope e request ID	Contrato estável e diagnóstico correlacionável
Saúde	GET /health	Permite health check do contêiner
Testes	Vitest + Supertest	Verifica API ponta a ponta com banco em memória
Entrega	Docker multi-stage	Imagem pequena e execução como usuário não root
CI	GitHub Actions	Executa npm ci e npm test em push e pull request
Documentação	OpenAPI 3.1	Contrato executável e explorável pelo parceiro

Cobertura comportamental dos testes

- Health check e presença do identificador de requisição.
- Ciclo completo criar, listar, buscar, contar, atualizar e excluir.
- Pesquisa parcial por nome sem diferenciar maiúsculas.
- Validação simultânea de nome, preço e estoque.
- Conflito de nomes duplicados e padronização de rota inexistente.

Evidência de execução

Vitest 5.0.1: 1 arquivo de teste aprovado, 6 testes aprovados e nenhuma falha. A mesma suíte é executada automaticamente pelo workflow de CI.

11. Execução e demonstração

Passos mínimos para o avaliador reproduzir a solução.

Execução local

```
npm ci
cp .env.example .env
npm run seed
npm start
```

Acessos

Recurso	Endereço
API	http://localhost:3000
Swagger UI	http://localhost:3000/docs
OpenAPI	http://localhost:3000/openapi.json
Health check	http://localhost:3000/health

Criação de exemplo

```
curl -X POST http://localhost:3000/api/v1/products \
-H "Content-Type: application/json" \
-d '{"name":"Notebook Pro 14","price":5499.90,"stock":15}'
```

Docker e testes

```
docker compose up --build

# Em outro terminal
npm test
```

Banco de demonstração

O comando `npm run seed` insere três produtos e é idempotente quanto aos nomes. O banco local fica em `data/products.db` e não é versionado. O Docker usa um volume persistente.

12. Decisões, limites e evolução

Trade-offs explícitos para demonstrar raciocínio arquitetural além da implementação.

Decisões registradas

Decisão	Motivo	Consequência
Node.js + Express	Aderência ao JavaScript e baixo custo de entrada	Ecossistema simples e API enxuta
SQLite nativo	Persistência sem serviço externo	Ótimo para avaliação e pequena escala
Service + Repository	Isolar negócio e infraestrutura	Mais arquivos, porém melhor evolução e teste
OpenAPI no código	Contrato disponível em runtime	Mudanças devem acompanhar endpoints
UUID gerado na aplicação	Identificador sem dependência do banco	Strings maiores que inteiros sequenciais

Limites conhecidos

- SQLite prioriza simplicidade e não é a opção final para muitas escritas concorrentes.
- A API acadêmica é pública e não implementa autenticação ou autorização por parceiro.
- A listagem ainda não possui paginação, filtros combinados ou ordenação selecionável.
- Preço usa número decimal em memória; ambiente financeiro crítico deve usar centavos inteiros ou tipo decimal do banco.

Evolução recomendada

Fase	Evolução
2	Autenticação OAuth2/JWT, rate limiting e autorização por escopo
3	PostgreSQL, migrações versionadas e paginação por cursor
4	Logs estruturados, métricas, tracing e SLOs
5	Cache, filas e eventos somente quando carga e casos de uso justificarem

13. Conclusão e checklist de entrega

Síntese do atendimento ao desafio e localização dos artefatos finais.

Conclusão

A Partner Products API demonstra os fundamentos solicitados: arquitetura documentada, MVC aplicado, responsabilidades isoladas, contrato REST completo e implementação funcional. Os diferenciais de persistência, testes, documentação executável, Docker e CI tornam a solução verificável e pronta para evolução.

Entregável	Arquivo / endereço	Situação
Relatório completo	docs/entregavel-final-arquitetura-software.pdf	Pronto
Arquitetura em PDF	docs/diagramas-arquitetura.pdf	Pronto
Arquitetura editável	docs/arquitetura.drawio	Pronto
Código e documentação	Repositório GitHub	Pronto
Persistência	SQLite + seed	Pronto
Testes	tests/products.e2e.test.js	6 aprovados

Repositório do projeto

Código-fonte e instruções

github.com/ronoellima10-ship-it/desafio-final-arquiteto-software-api

O README contém instalação, endpoints, exemplos, testes e links para todos os entregáveis.

Autor: Ronoel Lima

Data: 19 de setembro de 2026

Projeto: Desafio Final - Bootcamp Arquiteto(a) de Software